

```
import nltk
import numpy as np
import pandas as pd
```

```
from nltk.corpus import sentence_polarity
```

```
nltk.download('sentence_polarity')
```

```
[nltk_data] Downloading package sentence_polarity to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping corpora/sentence_polarity.zip.
True
```

```
all_positive_sentences = sentence_polarity.raw(categories='pos').strip().split('\n')
all_negative_sentences = sentence_polarity.raw(categories='neg').strip().split('\n')
```

```
all_positive_sentences[:3]
```

```
['the rock is destined to be the 21st century\'s new " conan " and that he\'s going to make a splash even greater than
arnold schwarzenegger , jean-claud van damme or steven segal . ',
 'the gorgeously elaborate continuation of " the lord of the rings " trilogy is so huge that a column of words cannot
adequately describe co-writer/director peter jackson\'s expanded vision of j . r . r . tolkien\'s middle-earth . ',
 'effective but too-tepid biopic']
```

```
len(all_positive_sentences)
```

```
5331
```

```
len(all_negative_sentences)
```

```
5331
```

```
test_pos = all_positive_sentences[4000:]
train_pos = all_positive_sentences[:4000]
test_neg = all_negative_sentences[4000:]
train_neg = all_negative_sentences[:4000]
```

```
len(train_neg)
```

```
4000
```

```
len(test_neg)
```

```
1331
```

```
train_x = train_pos + train_neg
test_x = test_pos + test_neg

train_y = np.append(np.ones(len(train_pos)), np.zeros(len(train_neg)))
test_y = np.append(np.ones(len(test_pos)), np.zeros(len(test_neg)))
```

```
len(train_x)
```

```
8000
```

```
train_x[7890]
```

```
'the biggest problem i have ( other than the very sluggish pace ) is we never really see her esther blossom as an actress ,
```

```
import re
import string

from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.tokenize import TweetTokenizer
```

```
nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
True
```

```
def process_sentence(sentence):
    """Process sentence function.
    Input:
        sentence: a string containing a sentence
    Output:
        sentences_clean: a list of words containing the processed sentence
    """
    stemmer = PorterStemmer()
    stopwords_english = stopwords.words('english')

    # remove hyperlinks
    sentence = re.sub(r'https?:\/\/[^\s\r\n]+', '', sentence)

    # tokenize sentences
    tokenizer = TweetTokenizer(preserve_case=False, strip_handles=True,
                               reduce_len=True)
    sentence_tokens = tokenizer.tokenize(sentence)

    sentences_clean = []
    for word in sentence_tokens:
        if (word not in stopwords_english and # remove stopwords
            word not in string.punctuation): # remove punctuation

            stem_word = stemmer.stem(word) # stemming word
            sentences_clean.append(stem_word)

    return sentences_clean
```

```
train_x[0]
```

```
'the rock is destined to be the 21st century\'s new " conan " and that he\'s going to make a splash even greater than arnol
```

```
process_sentence(train_x[0])
```

```
['rock',
 'destin',
 '21st',
 'century',
 'new',
 'conan',
 'go',
 'make',
 'splash',
 'even',
 'greater',
 'arnold',
 'schwarzenegg',
 'jean-claud',
 'van',
 'damm',
 'steven',
 'segal']
```

```
len(train_y)
```

```
8000
```

```
def build_freqs(sentences, ys):
    """Build frequencies.
    Input:
        sentences: a list of sentences
        ys: an m x 1 array with the sentiment label of each sentence
            (either 0 or 1)
    Output:
        freqs: a dictionary mapping each (word, sentiment) pair to its
            frequency
    """
    # Convert np array to list since zip needs an iterable.
    # The squeeze is necessary or the list ends up with one element.
    # Also note that this is just a NOP if ys is already a list.
    yslist = np.squeeze(ys).tolist()

    # Start with an empty dictionary and populate it by looping over all sentences
    # and over all processed words in each sentence.
    freqs = {}
    for y, sentence in zip(yslist, sentences):
        for word in process_sentence(sentence):
            pair = (word, y)
            if pair in freqs:
                freqs[pair] += 1
            else:
                freqs[pair] = 1
```

```
return freqs
```

```
freqs = build_freqs(train_x, train_y)
```

```
len(freqs)
```

```
17825
```

```
freqs[('film', 0)]
```

```
543
```

```
pos_top = sorted([(w, c) for (w, y), c in freqs.items() if y == 1], key=lambda t: -t[1])
neg_top = sorted([(w, c) for (w, y), c in freqs.items() if y == 0], key=lambda t: -t[1])
```

```
print("Слів у позитиві:", len(pos_top))
```

```
Слів у позитиві: 8772
```

```
print("Слів у негативі:", len(neg_top))
```

```
Слів у негативі: 9053
```

```
print("Позитив, найчастіші:", pos_top[:10])
```

```
Позитив, найчастіші: [('film', 696), ('movi', 469), ('. . .', 311), ('one', 272), ('make', 255), ('like', 230), ('perform',
```

```
print("Позитив, найрідші:", pos_top[-10:])
```

```
Позитив, найрідші: [('hubert', 1), ('slugfest', 1), ('toothless', 1), ('compress', 1), ('evanesc', 1), ('dubiou', 1), ('true
```

```
print("Негатив, найчастіші:", neg_top[:10])
```

```
Негатив, найчастіші: [('movi', 655), ('film', 543), ('like', 360), ('. . .', 312), ('one', 281), ('make', 215), ('character',
```

```
print("Негатив, найрідші:", neg_top[-10:])
```

```
Негатив, найрідші: [('screw', 1), ('acid', 1), ('all-mal', 1), ('swoon', 1), ('intermezzo', 1), ('constrict', 1), ('tighter'
```

```
def sigmoid(z):
    h = 1 / (1 + np.exp(-z))
    return h
```

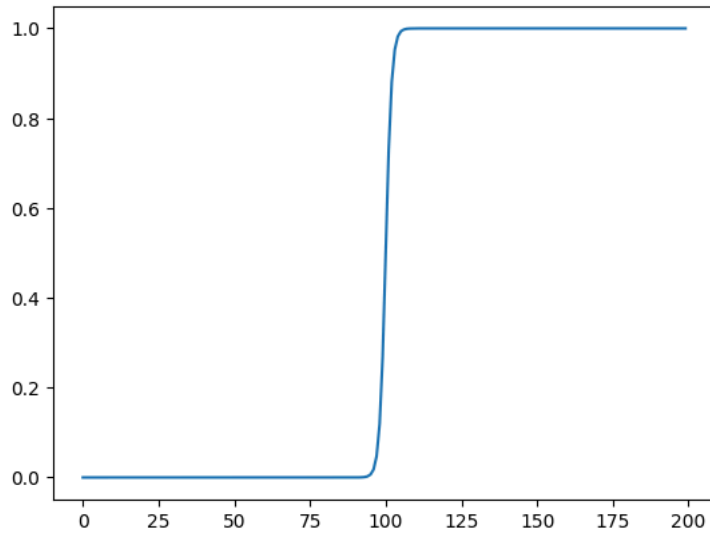
```
res = []
```

```
for z in range(-100, +100):
    res.append(sigmoid(z))
```

```
from matplotlib import pyplot as p
```

```
p.plot(res)
```

[<matplotlib.lines.Line2D at 0x7d25002056d0>]



```
def gradientDescent(x, y, theta, alpha, num_iters):
    m = x.shape[0]

    # Переконаємося, що y має правильну форму (m,1)
    if y.ndim == 1:
        y = y.reshape(-1, 1)

    for i in range(0, num_iters):
        # Обчислюємо гіпотезу
        z = np.dot(x, theta)
        h = sigmoid(z)

        # Обчислюємо функцію втрат - скалярне значення
        # Використовуємо поелементне множення і сумування
        epsilon = 1e-15 # Маленька константа для уникнення log(0)
        h_safe = np.clip(h, epsilon, 1 - epsilon)
        J = (-1/m) * np.sum(y * np.log(h_safe) + (1-y) * np.log(1 - h_safe))

        # Оновлюємо параметри
        theta = theta - (alpha/m) * np.dot(x.T, (h-y))

        # Виводимо значення помилки (опціонально)
        if i % 10 == 0:
            print(i, J)

    return J, theta
```

```
def extract_features(sentence, freqs):
    word_l = process_sentence(sentence)
    x = np.zeros((1, 3))
    x[0,0] = 1
    for word in word_l:
        if (word, 1.0) in freqs:
            x[0,1] += freqs[(word, 1.0)]
        if (word, 0.0) in freqs:
            x[0,2] += freqs[(word, 0.0)]
    return x
```

```
def predict_sentence(sentence, freqs, theta):
    x = extract_features(sentence, freqs)
    y_pred = sigmoid(np.dot(x, theta))
    return y_pred
```

freqs

('rather', 1.0): 33,

```
( 'substanc', 1.0): 9,
( 'appear', 1.0): 11,
( 'treebeard', 1.0): 1,
( "gollum", 1.0): 2,
( 'role', 1.0): 30,
( 'either', 1.0): 15,
( 'incred', 1.0): 12,
( 'ingeni', 1.0): 9,
( 'construct', 1.0): 15,
( 'memento', 1.0): 2,
( 'page-turn', 1.0): 2,
( "can't", 1.0): 36,
( 'wait', 1.0): 9,
( 'happen', 1.0): 15,
( 'next', 1.0): 11,
( 'hanek', 1.0): 5,
( 'challeng', 1.0): 18,
( 'confront', 1.0): 8,
( 'realiti', 1.0): 20,
( 'sexual', 1.0): 21,
( 'aberr', 1.0): 1,
( 'absorb', 1.0): 20,
( 'clariti', 1.0): 5,
( 'gone', 1.0): 8,
( 'benchmark', 1.0): 1,
( 'contend', 1.0): 1,
( 'measur', 1.0): 10,
( 'pain', 1.0): 19,
( 'viewer', 1.0): 30,
( 'chanc', 1.0): 12,
( "year'", 1.0): 16,
( 'accomplish', 1.0): 12,
( 'rivet', 1.0): 17,
( 'albeit', 1.0): 1,
( 'depress', 1.0): 10,
( 'view', 1.0): 31,
( 'iranian', 1.0): 8,
( 'rural', 1.0): 3,
( 'life', 1.0): 108,
( 'close', 1.0): 20,
( 'iraqi', 1.0): 1,
( 'imagin', 1.0): 41,
( 'comedi', 1.0): 171,
( 'thriller', 1.0): 56,
( 'artsi', 1.0): 1,
( 'flourish', 1.0): 3,
```

```
X = np.zeros((len(train_x), 3))
for i in range(len(train_x)):
    X[i, :] = extract_features(train_x[i], freqs)
```

X.shape

(8000, 3)

X[7995:]

```
array([[ 1., 895., 728.],
       [ 1., 481., 452.],
       [ 1., 611., 786.],
       [ 1., 572., 574.],
       [ 1., 178., 287.]])
```

train_x[7995:]

```
['a nearly 21/2 hours , the film is way too indulgent . ',
 'gorgeous to look at but insufferably tedious and turgid . . . a curiously constricted epic . ',
 'it looks much more like a cartoon in the end than the simpsons ever has . ',
 'with a tighter editorial process and firmer direction this material could work , especially since the actresses in the
 lead roles are all more than competent , but as is , personal velocity seems to be idling in neutral . ',
 "doesn't really add up to much . "]
```

Y = train_y.reshape(-1, 1)

Y.shape

(8000, 1)

Пояснення до п. 9: шляхом зміни швидкості навчання та кількості ітерацій були виявлені оптимальні параметри: 1e-3 для швидкості навчання, 1250 для кількості ітерацій. Якщо далі збільшувати швидкість навчання (наприклад, 0.01), то помилка зростає.

```
1 theta = gradientDescent(X, Y, np.zeros((3, 1)), 1e-3, 1250)
```

```
print(f"Помилка після навчання: {J:.8f}.")
print(f"Вектор вагових коефіцієнтів: {[round(t, 8) for t in np.squeeze(theta)]}")
```

```
710 10.517323504709507
720 10.51731622291722
730 10.517308448203652
740 10.517304697912978
750 10.51729783694427
760 10.517291573437404
770 10.517283727742056
780 10.517277353811082
790 10.517268792600849
800 10.517261313600077
810 10.517254808501402
820 10.517248172843267
830 10.517241566604447
840 10.517234852129015
850 10.517219906334097
860 10.517210473551804
870 10.517202923162845
880 10.517194849129082
890 10.517186995331368
900 10.517180499398838
910 10.517172349696162
920 10.517165652535295
930 10.517159232854866
940 10.517152839831757
950 10.517145514715587
960 10.517138906976033
970 10.517131459269669
980 10.517124754089842
990 10.517118437400166
/tmp/ipykernel_2449/4005796728.py:2: RuntimeWarning: overflow encountered in exp
  h = 1 / (1 + np.exp(-z))
1000 10.517105646429641
1010 10.517103295510628
1020 10.517096361483079
1030 10.517090154208644
1040 10.517083585154896
1050 10.517077206377012
1060 10.517070421949496
1070 10.517060639401722
1080 10.517053923761015
1090 10.517047571482898
1100 10.517040243269836
1110 10.517030762920156
1120 10.517023684233278
1130 10.517018597205078
1140 10.517011952887916
1150 10.517005361070105
1160 10.516999502217688
1170 10.517000331221123
1180 10.516993209489247
1190 10.51698518840453
1200 10.516977954804533
1210 10.516971253789759
1220 10.516963775067822
1230 10.516956737131453
1240 10.516949991025887
Помилка після навчання: 9.52105172.
Вектор вагових коефіцієнтів: [np.float64(-0.02293684), np.float64(0.85910808), np.float64(-0.66179039)]
```

```
sentence = " ".join(neg_sentences[0])
sentence
```

```
'simplistic , silly and tedious .'
```

```
if predict_sentence(sentence, freqs, theta) >= 0.5:
    print("позитив")
else:
    print("негатив")
```

```
негатив
```

```
def test_logistic_regression(test_x, test_y, freqs, theta):
    y_hat = []
    for sentence in test_x:
        # Якщо елемент у test_x є списком слів, об'єднуємо його у рядок перед прогнозом
        if isinstance(sentence, list):
            sentence = " ".join(sentence)

        y_pred = predict_sentence(sentence, freqs, theta)
        if y_pred > 0.5:
            y_hat.append(1)
        else:
            y_hat.append(0)
```

```
accuracy = (np.sum(np.array(y_hat) == np.squeeze(test_y))) / len(test_x)
return accuracy
```

```
tmp_accuracy = test_logistic_regression(test_x, test_y, freqs, theta)
print(f"Точність логістичної регресії: {tmp_accuracy:.4f}")
```

Точність логістичної регресії: 0.6146

```
test_msg = "A wonderful movie with brilliant acting, a fantastic storyline, and deeply moving emotional scenes."
```

```
p = predict_sentence(test_msg, freqs, theta)
if p > 0.65:
    print("позитив", p)
else:
    print("негатив", p)
```

позитив [[1.]]

```
test_msg = "This film has incredible acting and a brilliant storyline, making it an absolute masterpiece of cinema."
```

```
p = predict_sentence(test_msg, freqs, theta)
if p > 0.65:
    print("позитив", p)
else:
    print("негатив", p)
```

позитив [[1.]]

```
my_sentence = "terrible plot and extremely boring actors"
```

```
print(process_sentence(my_sentence))
y_hat = predict_sentence(my_sentence, freqs, theta)
print(y_hat)
```

```
if y_hat > 0.5:
    print('Позитивна тональність')
else:
    print('Негативна тональність')
```

['terribl', 'plot', 'extrem', 'bore', 'actor']
[[9.9249677e-32]]
Негативна тональність

result = [freqs, theta]

```
type(freqs)
```

dict